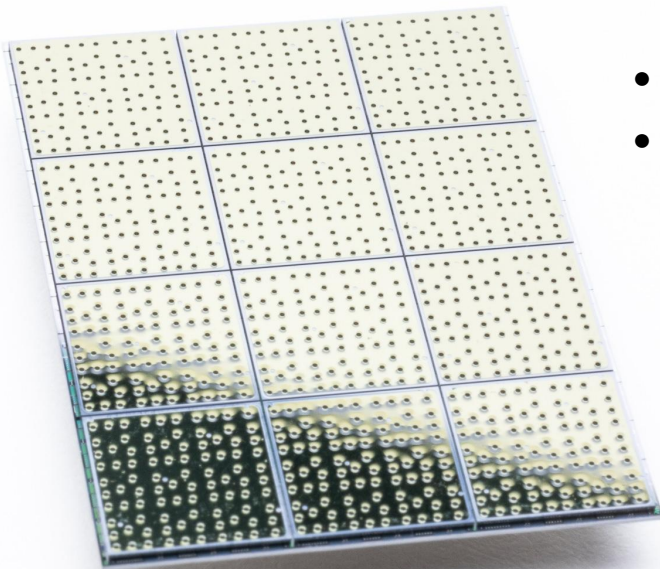


Cepheus™-1-108Q

Overview:

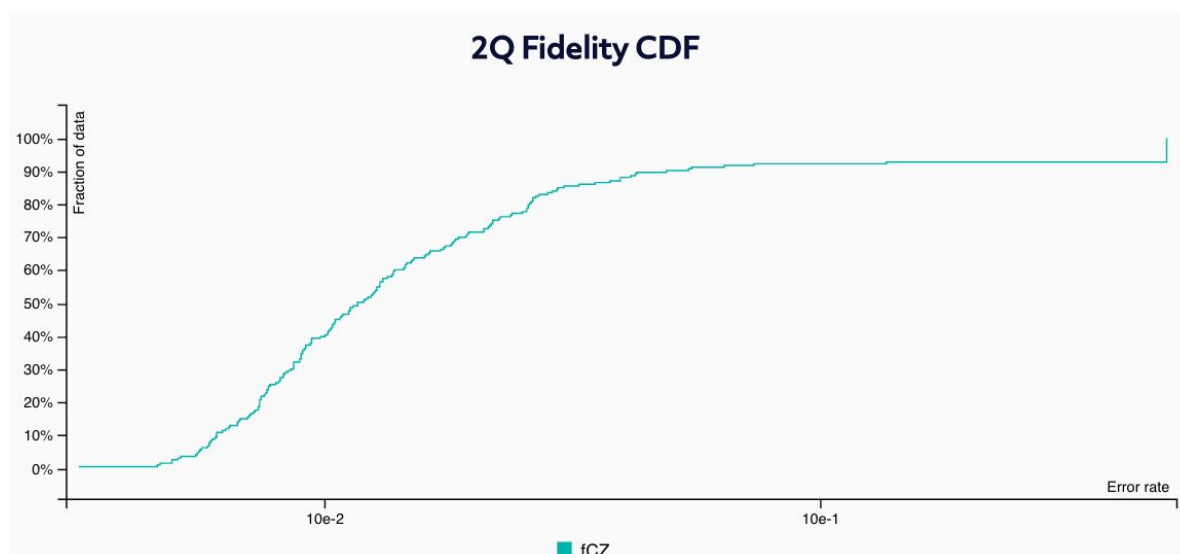
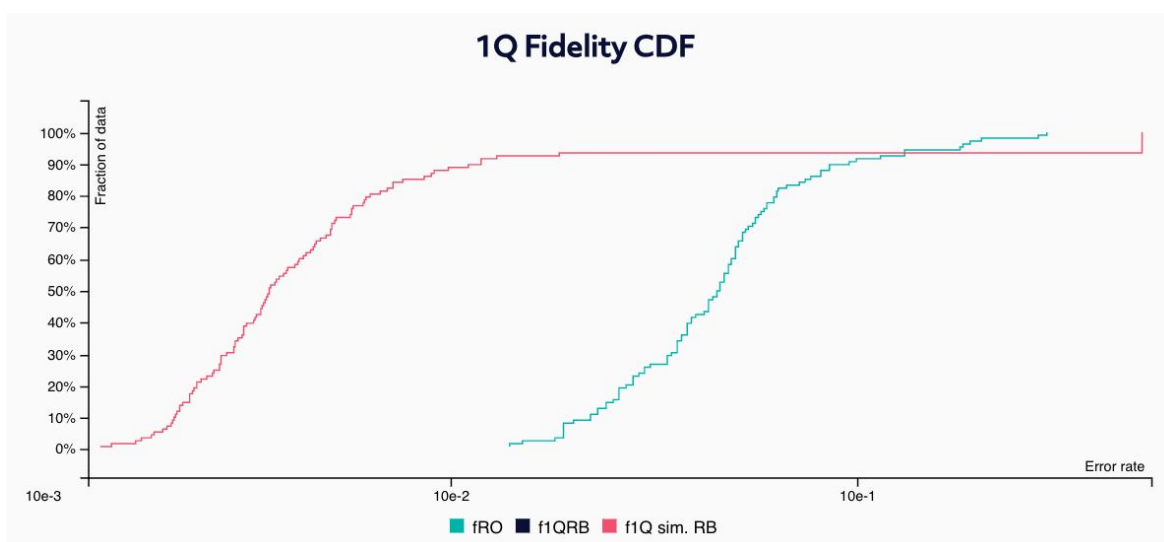
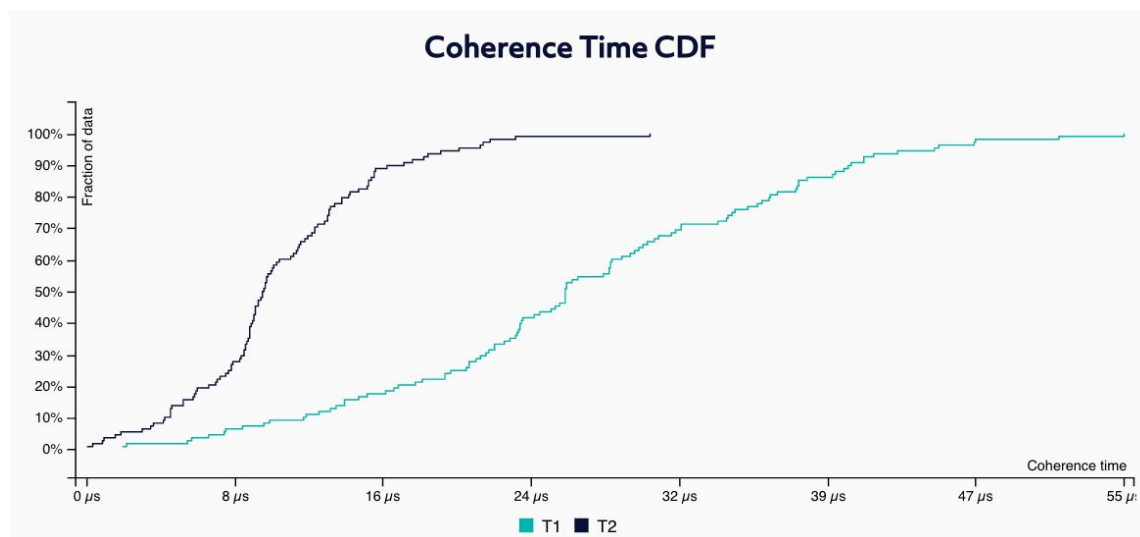
- **Qubits:** 108 tunable-frequency transmon qubits
- **Coupling:** 216 tunable couplers
- **Architecture:** 4x3 array of 9-qubit chiplets with inter-module couplers, 4-fold connectivity
- **Measurement:** 6:1 multiplexed readout resonators with single-shot dispersive readout
- **Native Gates:** R_x , R_y , R_z , CZ
- **Programming model:** pyQuil SDK, gate-level via Quil, pulse-level via Quil-T



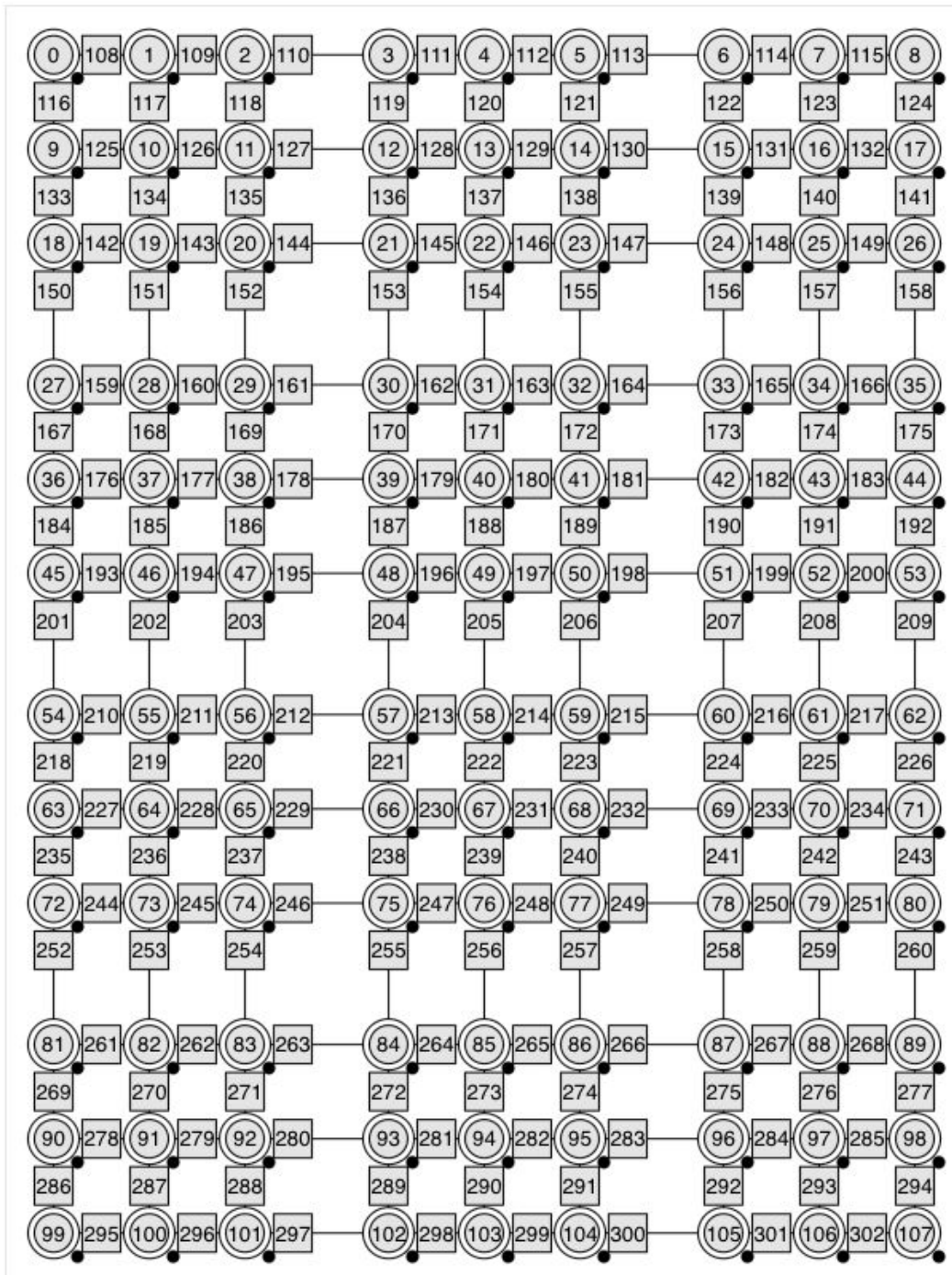
Performance Metric	Median Value
1Q gate error (isolated)	0.13%
1Q gate error (simultaneous)	0.4%
2Q gate error (CZ)	0.9%
1Q gate speed	40 ns
2Q gate speed	60 ns
Readout error (simultaneous)	4.5%
T_1 lifetime	25 μ s
T_2 lifetime	10 μ s

Performance Metrics - Cumulative Distribution Functions:

(Interactive CDF plots available at <https://qcs.rigetti.com/qpus>)



QPU Topology:

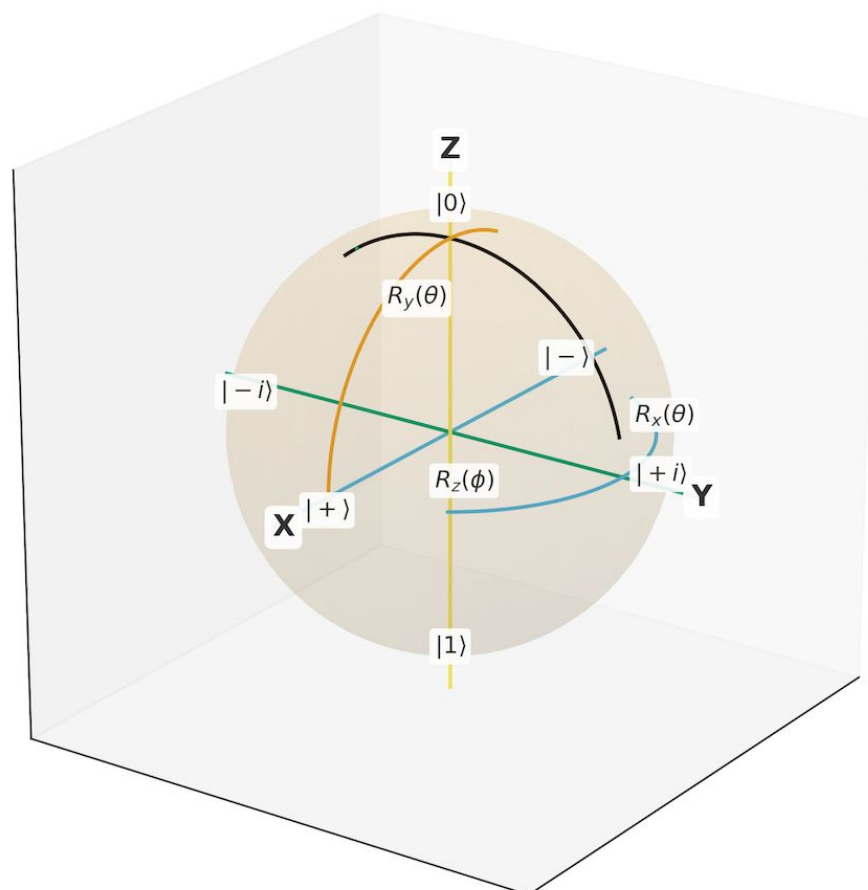


○ = Qubit
 □ = Tunable Coupler

Instruction Set Architecture - Gates and Pulses

(part 1)

- **Repetition rate** = 50 kHz
- **Single-qubit gates:**
 - 1Q gate speed = 40 ns
 - **$R_X(\theta)$** : a rotation by an angle $\theta = \pi$ or $\theta = \pm\pi/2$ about the x-axis of the Bloch sphere
 - **$R_Y(\theta)$** : a rotation by an angle $\theta = \pi$ or $\theta = \pm\pi/2$ about the y-axis of the Bloch sphere
 - **$R_Z(\theta)$** : a rotation by an angle θ about the z-axis of the Bloch sphere
 - **Note:** The RZ gate is implemented with a local update of in-sequence phases that consumes zero runtime and introduces negligible error.



Instruction Set Architecture - Gates and Pulses

(part 2)

- **Two-qubit gates:**

- 2Q gate speed = 60ns
- **CZ(control, target):** a CPHASE operation applied with the rotation parameter $\theta = \pi$. A Z-gate is applied to the target qubit if the control qubit is in the excited state.
 - **Note:** In contrast to CPHASE, CZ offers single-pulse construction, allowing for lower error rates when full phase control is not needed.

- Additional non-native gates can be performed – a comprehensive list is available at: <https://pyquil-docs.rigetti.com/en/latest/apidocs/pyquil.gates.html>

- **Pulse-level waveform customization:**

- Custom pulse and capture operations can be specified using Quil-T.
 - Enables direct control over the baseband waveform " $u(t)$ ", run-time frequency " f ", phase " θ ", and scale " α " of the signal, which has the following general form:

$$x(t) = \text{Re}[\alpha u(t) e^{i(\theta + 2\pi f t)}]$$

- See <https://pyquil-docs.rigetti.com/en/latest/quilt.html> for more details on using Quil-T for custom baseband waveforms.

Instruction Set Architecture - Quil Syntax:

Instruction	Support	Notes
ADD, SUB	Supported for INTEGER and REAL; wraps on overflow.	Prior to Nov 2024, ADD/SUB were unintentionally supported on BIT. Replacement instruction on BIT is XOR.
CAPTURE	Supported	See waveform details in Quil-T (CAPTURE uses device waveforms).
DECLARE	Supported	BIT, OCTET, INTEGER as signed 48-bit. REAL as s22.22 fixed-point
DELAY	Full support	—
DIV, MUL	Supported; wraps on overflow.	Operands must share the same data type; latter operand must be a literal power of two with magnitude $< 2^{47}$
FENCE	Supported	—
LOAD, STORE	Supported	Dynamic offset is bounds-checked at runtime for memory safety; out-of-bounds terminates program with a fixed error code (region/offset not included in error code).
RAW-CAPTURE	Supported (with caveats)	1 per program; cannot appear with CAPTURE; has a max duration (implementation-dependent). Inserts a tail delay for readout transfer proportional to instruction duration. Reads a group of qubits ; look for frame attribute ENABLE-RAW-CAPTURE: "true" to infer qubit grouping.
SET-FREQUENCY, SHIFT-FREQUENCY	Supported	Units: Hertz .
SET-PHASE, SHIFT-PHASE	Supported	Units: Radians .
SET-SCALE	Supported	Unitless factor s with $-4 \leq s \leq 4$. Parametric values also supported.
SWAP-PHASES	Supported	—
MOVE	Supported	Between memory regions of the same data type .
AND, IOR/OR, XOR	Supported	For INTEGER and BIT.
EQ, GE, GT, LE, LT	Supported	Between regions of the same data type .
NEG, NOT	Supported	For INTEGER and BIT.
NOP	Supported	Has no effect on the program.
CONVERT	Not supported	—
LABEL, JUMP, JUMP-WHEN, JUMP-UNLESS, HALT	Supported	Jump condition operand may be any Quil datatype, not only BIT. Each starts a new basic block; no pulse-timing guarantees across the boundary.
EXCHANGE	Not supported	—

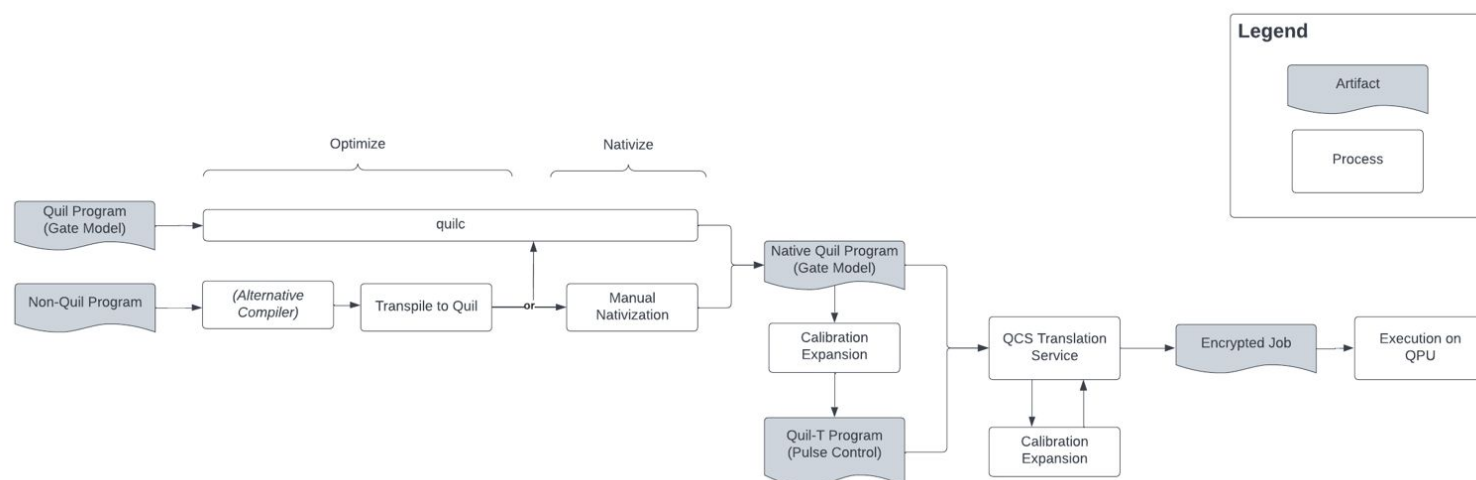
Compilation and Toolchain:

- **Quil compiler:**

- An optimizing compiler that translates arbitrary Quil → native Quil → executables
- Quil programs themselves may be gate-model, pulse-model, or a combination of the two.
- Quil is capable of *dynamic control flow* to allow programs to specify conditions using memory values, enabling them to dynamically branch using run-time values.
 - More details at <https://docs.rigetti.com/qcs/guides/quil/dynamic-control-flow>

- **Parametric compilation:**

- Allows for compilation of initial program ansatz just once.
- A fixed template of gates is iteratively updated with new input parameters, determined by a classical optimizer.
- New parametric gates can be constructed inline to Quil programs as long as a matrix representation of the gate is known.
- The below diagram summarizes the workflows available to a Quantum Cloud Services (QCS) user running a program on a live QPU:



- **Additional integrations available:**

- QCS also supports programs written in other languages via transpilation to Quil:
 - Rust - package information available at <https://github.com/rigetti/quil-rs>
 - QIR - package information available at <https://github.com/rigetti/qcs-sdk-qir>
 - Qiskit - package information available at <https://github.com/rigetti/qiskit-rigetti>

rigetti